Journal of Machine Learning for Modeling and Computing, 5(1):39–67 (2024)

PHYSICS-INFORMED NEURAL NETWORKS FOR MODELING OF 3D FLOW THERMAL PROBLEMS WITH SPARSE DOMAIN DATA

Saakaar Bhatnagar,* Andrew Comerford, & Araz Banaeizadeh

Altair Engineering Inc., 100 Mathilda Place, Sunnyvale, CA, USA

*Address all correspondence to: Saakaar Bhatnagar, Altair Engineering Inc., 100 Mathilda Place, Sunnyvale, CA, 94086, USA, E-mail: sbhatnagar@altair.com

Original Manuscript Submitted: 11/3/2023; Final Draft Received: 3/5/2024

Successfully training physics-informed neural networks (PINNs) for highly nonlinear partial differential equations (PDEs) on complex 3D domains remains a challenging task. In this paper, PINNs are employed to solve the 3D incompressible Navier–Stokes equations at moderate to high Reynolds numbers for complex geometries. The presented method utilizes very sparsely distributed solution data in the domain. A detailed investigation of the effect of the amount of supplied data and the PDE-based regularizers is presented. Additionally, a hybrid data–PINNs approach is used to generate a surrogate model of a realistic flow thermal electronics design problem. This surrogate model provides near real-time sampling and was found to outperform standard data-driven neural networks (NNs) when tested on unseen query points. The findings of the paper show how PINNs can be effective when used in conjunction with sparse data for solving 3D nonlinear PDEs or for surrogate modeling of design spaces governed by them.

KEY WORDS: *physics-informed neural networks, Navier–Stokes equations, surrogate modeling, machine learning, design optimization*

1. INTRODUCTION

Over the last few years, there has been significant growth in the popularity of machine learning (ML) algorithms to solve partial differential equations (PDEs) or assist PDE solvers, such as computational fluid dynamics (CFD) solvers (Brunton et al., 2020; Calzolari and Liu, 2021). A particular application where CFD solvers struggle, due to the computational cost, is iterative design optimization. This is the process of continually updating a design (e.g., an electronics assembly layout) and computing the solution (e.g., flow or thermal fields) to optimize the performance (e.g., constrain the temperatures or reduce the pressure drop). The challenge for CFD is that the input–output relationship is one-to-one. Therefore, any changes to the input vector (e.g., geometric variations) need to be resimulated, leading to high costs when iterating on different design scenarios. Overall high-fidelity iterative design requires a prohibitive level of resources, both computationally and monetarily, and often leads to a suboptimal outcome. The attraction of ML algorithms in these scenarios is the ability to rapidly find solutions for such problem setups that are challenging in conventional CFD, such as large design space explorations (Liu et al.,

2689–3967/24/$35.00

2022), turbulence model closure (Zhao et al., 2020), or solving incomplete/ill-posed problems (Cai et al., 2021c).

Conventional ML algorithms usually require large amounts of data to train. This represents a challenge when using ML in engineering applications such as CFD, since experimental data can be difficult and expensive to obtain and may suffer from measurement noise. Furthermore, in many engineering experiments, field data such as temperature and velocity fields can sometimes only be captured at specific locations, and it is difficult to get full field solution results from physical experiments. Research has turned to using simulation data for training ML models, but the computational cost of generating large amounts of data to train models is a major bottleneck.

Physics-informed neural networks (PINNs) (Raissi et al., 2019) represent an advance in scientific ML that has the potential to solve many of the aforementioned issues. By adding the physics that governs the problem into the loss function, and optimizing the loss, it is possible to have the network learn the solution of the problem represented by that equation in a data-free manner. PINNs can be used in cases where sporadic experimental field data is available (Cai et al., 2021b; Jagtap et al., 2022) to calculate the rest of the field variable and can be used to solve problems with incomplete or missing physics (Chen et al., 2020; Ishitsuka and Lin, 2023).

Another application area in which PINNs could be very beneficial is ML–based surrogate modeling. Although a relatively new field, several ML architectures and methods have been utilized in the literature. These include proper orthogonal decomposition (POD) (Krath et al., 2021), gappy POD (Nekkanti and Schmidt, 2023), and manifold learning (Melas-Kyriazi, 2020). More recently, increased attention has been given to statistical methods like Gaussian processes and neural networks (NNs) that incorporate ML to create surrogate models. Bhatnagar et al. (2019) used a convolutional neural network (CNN) architecture to predict aerodynamic flow fields over airfoils and created a surrogate model that generalized between flow conditions and airfoil geometries. Guo et al. (2016) also used a CNN architecture to predict steady flows over automotive vehicles. Lee and You (2019) used generative adversarial networks (GANs) coupled with physical laws to predict unsteady flow around a cylinder, demonstrating the benefits of using embedded physics. Raissi and Karniadakis (2018) used Gaussian processes to model and identify several complex PDEs.

Several of the aforementioned studies used purely data-driven models and required the creation of large amounts of training data to generate accurate and generalizable models. PINNs have the capability to greatly reduce these data generation costs, and it has been shown that training surrogates using the physics embedded in the loss function greatly improves predictive accuracy across a wide range of applications (Antonello et al., 2023; Kim et al., 2023; Lee and You, 2019; Wang et al., 2021a).

However, there is currently a lack of research articles applying PINNs to three-dimensional (3D) problems, particularly for highly nonlinear PDEs like the Navier–Stokes equations. These problems are challenging for PINNs due to a variety of reasons that are discussed later in this paper. Yet, these problems are the most lucrative to solve, as most industrial applications of CFD are done in 3D. This paper provides results that aim to address this gap by solving several problems with realistic physical parameters, over complex geometries in a data-assisted manner, using very sparse domain data. Further, this paper solves a realistic flow thermal design optimization problem using a hybrid data–PINN surrogate model and shows how PINN models outperform standard data-driven NN surrogates for every test point queried in the design of experiments (DoE) space for the surrogate modeling problem.

The paper is divided as follows. Section 2 introduces PINNs in more detail and discusses some of the technical challenges with training PINNs. Section 3 outlines some of the important

features the authors incorporate in the creation and training of PINNs to enable accurate and fast convergence. Section 4 demonstrates several problems solved using PINNs and showcases a design optimization problem using PINN-based surrogates. Section 5 discusses how the work shown in this paper can be improved upon.

2. PHYSICS-INFORMED NEURAL NETWORKS (PINNs)

2.1 Setting Up a PINN Training

PINNs leverage automatic differentiation to obtain an analytical representation of an output variable and its derivatives, given a parametrization using the trainable weights of the network. By employing the underlying static graph, it is possible to construct the differential equations that govern physical phenomena.

A PDE problem in the general form reads:

$$\mathcal{N}_{\mathbf{x}}[u] = 0, \quad \mathbf{x} \in \Omega, \tag{1}$$

$$\Phi(u(\mathbf{x})) = \mathbf{g}(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega, \tag{2}$$

where Φ can be the identity operator (Dirichlet boundary condition) or a derivative operator (Neumann/Robin boundary condition). In order to solve the PDE using the PINN method, the residual of the governing PDE is minimized, which is defined by

$$r_\theta(\mathbf{x}) = \mathcal{N}_{\mathbf{x}}[f_\theta(\mathbf{x})], \tag{3}$$

where f_θ is the predicted value by the network. The residual value, along with the deviation of the prediction from boundary/initial conditions, is used to construct the loss, which takes the form

$$L(\theta) = L_r(\theta) + \sum_{i=1}^{M} \lambda_i L_i(\theta), \tag{4}$$

where the index i refers to different components of the loss function, relating to initial conditions, boundary conditions, and measurement/simulation data. λ_i refers to the weight coefficient of each loss term. The individual loss terms are constituted as follows:

$$L_r = \frac{1}{N_r}\sum_{i}^{N_r}\left[r(\mathbf{x}_r^i)\right]^2, \quad L_b = \frac{1}{N_b}\sum_{i}^{N_b}\left[\Phi(\hat{u}(\mathbf{x}_b^i)) - g_b^i\right]^2, \quad L_d = \frac{1}{N_d}\sum_{i}^{N_d}\left[u(\mathbf{x}_d^i) - \hat{u}(x_d^i, t_d^i)\right]^2, \tag{5}$$

where the subscripts r, b, and d refer to collocation, boundary, initial, and data points, respectively. The loss term $L(\theta)$ can then be minimized to have the network learn the solution to the PDE described by Eqs. (1) and (2). A popular method is to use gradient-based optimizers like Adam (Kingma and Ba, 2014) and L-BFGS to optimize the network weights.

2.2 Current Challenges with PINNs

Although the PINN method shows great promise, it still has a number of unresolved issues. The biggest challenges with PINNs currently lie in the scalability of the algorithms to large 3D problems as well as problems with complex nonlinearities and unsteady problems. Some of the issues described henceforth are tackled by methods described in Section 3.

2.2.1 Weak Imposition of Boundary Conditions

The solution of a PDE problem must obey all initial and boundary conditions imposed on it while minimizing the residual of the governing equation. However, for NN–based solvers, it is difficult to impose boundary and initial conditions in an exact manner. This is because the standard way to impose boundary conditions in PINNs is to create a linear combination of loss functions (as described mathematically in the previous section). Each loss either describes the deviation of the network output from a specific boundary condition or the magnitude of the residual of the governing equations. Therefore, boundary conditions are only satisfied in a weak manner. There has been research demonstrating the utility of exact imposition of boundary conditions (Liu et al., 2019; Sukumar and Srivastava, 2022; Sun et al., 2020) or creative multi-network approaches (Rao et al., 2021); such implementations are mostly problem specific and do not generalize well.

Weak imposition of boundary conditions also creates another issue, one that is fairly common in multi-task learning and multi-objective optimization: choosing the values of loss term coefficients that make up the linear combination. Choosing these weights is a nontrivial exercise that would require calibration via hyperparameter search, which is not feasible. Wang et al. (2021b) introduced a heuristic dynamic weighting algorithm to update and select these weights automatically and continuously during the training to enable convergence to the correct answer. Additionally, there have been several other algorithms proposed to choose the correct scheme for weighting the losses (Bischof and Kraus, 2021; Wang et al., 2022a,b). This continues to be an active area of research in the PINNs community. Finally, methods have been proposed to impose the boundary conditions in a strong manner by manipulating the output formulations (Sun et al., 2020) or by utilizing operator networks (Saad et al., 2023).

2.2.2 Difficult Optimization Problem

A second problem is the nature of the loss landscape itself, in which a reasonable local minimum is required to be found. As seen in Krishnapriyan et al. (2021), Gopakumar et al. (2023), Subramanian et al. (2022), and Basir and Senocak (2022), as well as the author's own experiments, different nondimensional quantities (e.g., Reynolds number) in the governing equations, the number of dimensions of the problem, the point cloud/discretization, the boundary conditions, and the complexity of the solution to be predicted can adversely affect the loss landscape of the NN training. This makes optimization challenging and can fail to find an adequate local minimum via a gradient descent-based algorithm. Recently, methods borrowing concepts from optimization theory have shown that alternate formulations (e.g., augmented Lagrangian method for the loss functions) can aid the convergence properties of the training problem (Basir and Senocak, 2022; Son et al., 2023). There have also been efforts toward imposing physical constraints in an integral form (Hansen et al., 2024).

2.2.3 Cost of Training

Constructing the PDE loss functions involves several backward passes through the network, which is a costly operation. PINNs on average take longer to train than their data-driven counterparts for exactly this reason; the computation graph of a PINN training is much more complex. Moreover, for the Navier–Stokes equations, it has been seen that although the stream function formulation provides better results (due to exact enforcement of continuity), it is costlier in terms of training time. As seen in NVIDIA's experiments (Hennigh et al., 2021), it can take several

million iterations for the more complex problems to be solved via PINNs. To reduce the cost of training, approaches such as automatic differentiation for finite difference formulations (He and Pathak, 2020), or using first-order formulations (Gladstone et al., 2022), have been proposed. However, these solutions tend to be mostly problem specific and do not necessarily generalize well to increased problem complexity and grid definitions. Meta-learning algorithms (Finn et al., 2017) have also recently gained significance as an effective way to reduce the cost of training NNs on new tasks, and some of this work has been extended to PINNs (Penwarden et al., 2023) as well.

3. IMPORTANT FEATURES FOR CREATING PINN MODELS

In this section, the important techniques used to create PINN-based models cost-effectively are outlined. The PINN models in subsequent sections are created by combining these features that have been found to have an effect on the accuracy of the model and the speed of training.

3.1 Hybrid Data-Physics Training

Compared with the original PINNs method proposed by Raissi et al. (2019), a plethora of research has been undertaken to improve and expand on the method (Cai et al., 2021a; Cuomo et al., 2022). From these developments, the PINNs method has been applied to solve PDE-based problems of increasing complexity and dimensionality. However, the PINNs method is currently not suited for solving engineering problems often encountered in industry in a data-free manner. The optimization issues and cost of model training outlined above make the method, presently, unsuitable for use as a forward solver. To get the best of both worlds, the PINNs method can be augmented with data. Figure 1 depicts the tradeoff between using only data or only physics, and that the sweet spot lies in using both. In addition to the discussed benefit of hybrid data-physics training reducing the cost of generating data, there have been several examples showing that the inclusion of sparse solution data in the training loss function significantly improves the convergence capabilities of the PINNs method (Cai et al., 2021a; Gopakumar et al., 2023; Sharma et al., 2023).

In this paper, we take inspiration from this and use sparse solution data to solve 3D flow thermal problems and inform our surrogate models with physics while creating them.

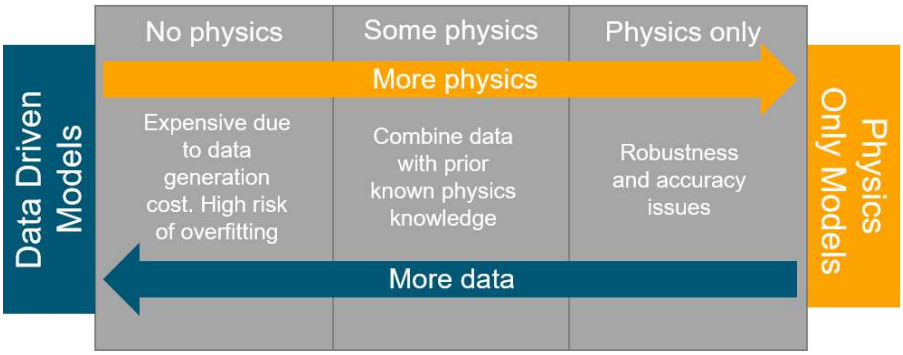


FIG. 1: The spectrum of data-driven versus physics-informed models. Incorporating governing physics information into the models during creation serves as an effective form of regularization and often helps reduce the amount of data required to achieve the same accuracy levels.

3.2 Modified Learning Rate (LR) Annealing

As described in Section 2.2.1, the learning rate (LR) annealing algorithm has proved to be effective in mitigating the stiffness of the PINN training problem. However, utilizing this method over a broader spectrum of problems highlighted an issue with stability. The following outlines this issue:

As shown in Eq. (4) the PINN loss function being optimized takes the form

$$L(\theta) = L_r(\theta) + \sum_{i=1}^{M} \lambda_i L_i(\theta). \tag{6}$$

At any training step, the update to the loss coefficient is calculated (Wang et al., 2021b) as

$$\hat{\lambda}_i = \frac{\max_\theta |\nabla_\theta L_r(\theta)|}{\overline{|\nabla_\theta L_i(\theta)|}}, \quad i = 1, \ldots, M.$$

It can be seen that if the loss L_i decreases much faster than L_r during the training, the value of $\hat{\lambda}_i$ increases. This then leads to a larger coefficient for that loss term and an associated faster decay of the loss.

This instability has the unintended consequence of the optimizer getting stuck in minima, where it minimizes the loss L_i very well but is unable to optimize for the loss of the other constraints. The proposed updated algorithm to mitigate this issue is shown in Algorithm 1. The values of thresholds are hyperparameters, but if the inputs and outputs of the network have been normalized (using standard score normalization, for example), then selecting values between 10^{-3} and 10^{-5} works well in practice.

Algorithm 1: Modified Learning Rate Annealing

---

for update step $= 1$ to N **do**

if $L_i(\theta) \leq$ (threshold)$_i$

$$\hat{\lambda}_i = 0$$

else

Compute $\hat{\lambda}_i$ by

$$\hat{\lambda}_i = \frac{\max_\theta |\nabla_\theta L_r(\theta)|}{\overline{|\nabla_\theta L_i(\theta)|}}, \quad i = 1, \ldots, M$$

end if

Update weights λ_i as

$$\lambda_i = (1 - \alpha)\lambda_i + \alpha\hat{\lambda}_i$$

Update network parameters via gradient descent:

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta L_r(\theta) - \eta \sum_{i=1}^{M} \lambda_i \nabla_\theta L_i(\theta)$$

end for

We set the hyperparameter $\alpha = 0.1$ and $\eta = 10^{-3}$. Threshold values are chosen somewhere between 10^{-3} and 10^{-5}.

For a problem with the loss function:

$$L(\theta) = L_r(\theta) + \lambda_{\text{neu}} L_{\text{neu}}(\theta) + \lambda_{\text{dir}} L_{\text{dir}}(\theta), \tag{7}$$

where $L_r(\theta)$, $L_{\text{neu}}(\theta)$, and $L_{\text{dir}}(\theta)$ correspond to the PDE, Neumann, and Dirichlet loss, respectively. Figure 2 shows the training curves for the individual losses and the value of the adaptive coefficients when they are calculated using Algorithm 1. It can be seen that when the boundary loss terms in Figs. 2(c) and 2(d) go below their thresholds (set to 10^{-5}), the associated coefficients shown in Figs. 2(a) and 2(b) start decaying. Following this, the PDE loss in Fig. 2(e) starts improving much faster. If the term $L_i(\theta)$ goes above its threshold, it leads to a spike in the adaptive constant λ_i, which brings it down again. Figure 2(f) also shows the PDE loss when using regular LR annealing, and it can be seen that the PDE loss stagnates quickly. The added time cost of using the modified LR annealing is the same as standard LR annealing, with each iteration taking 1.5× to 2.2× longer than using fixed weights.

3.3 Fourier Feature Embeddings

As described in Tancik et al. (2020), artificial neural networks (ANNs) suffer from a spectral bias problem. To overcome this, they introduced a Fourier feature embedding that allows models to effectively capture high-frequency components of the solution. This has the effect of markedly improving the ability of the networks to capture sharp gradients in the solutions, which requires the network to be able to learn high-frequency components of the solution quickly.

Following the implemenation in Tancik et al. (2020), for an input vector

$$\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix},$$

instead of using $\mathbf{v}$ as the input, we compute the Fourier feature mapping:

$$\gamma(\mathbf{v}) = \left[\cos(2\pi \mathbf{b}_1^T \mathbf{v}),\, \sin(2\pi \mathbf{b}_1^T \mathbf{v}),\, \ldots,\, \cos(2\pi \mathbf{b}_m^T \mathbf{v}),\, \sin(2\pi \mathbf{b}_m^T \mathbf{v})\right], \tag{8}$$

where m is a hyper parameter, and the frequencies $\mathbf{b}_j^T$ are selected randomly from an isotropic distribution. Then $\gamma(\mathbf{v})$ is passed into the network.

The Fourier feature embedding was shown to be highly effective in training PINN models by Wang et al. (2021c), and several results were shown for 1D and 2D problems. We extend this implementation to solve 3D flow problems via PINNs and use it to create our hybrid data–PINN surrogate for flow thermal problems. In Appendix A.1.2, the difference in predicted solution for a flow problem with and without the Fourier feature embedding is shown.

In addition, there have been other proposed solutions for the spectral bias problem for applications to PDE problems, such as the Siren activation (Sitzmann et al., 2020), Fourier neural operators (Li et al., 2020), and weighting schemes derived from the theory of neural tangent kernels (NTK) (Wang et al., 2022b).

4. EXPERIMENTS AND RESULTS

In this section, some example problems are solved using PINNs. Sections 4.1 and 4.2 solve the 3D incompressible Navier–Stokes equations through a data-assisted approach, where sparse solution data is provided in the domain.

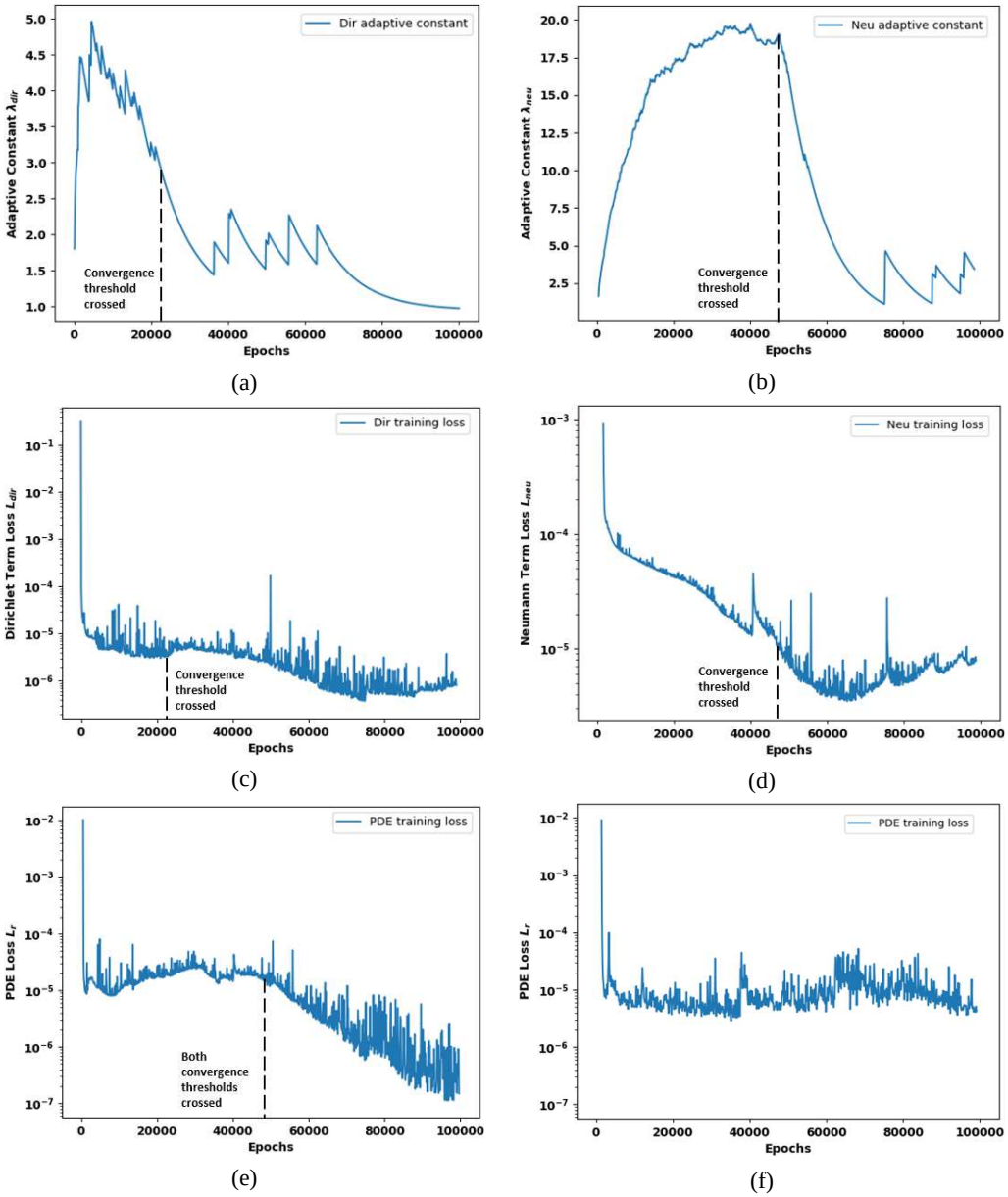


FIG. 2: Adaptive coefficients and loss terms from Eq. (7) during training. (a) Evolution of the Dirichlet loss adaptive constant during training. (b) Evolution of the Neumann loss adaptive constant during training. (c) Dirichlet boundary condition loss term $L_{\text{dir}}(\theta)$. (d) Neumann boundary condition loss term $L_{\text{neu}}(\theta)$. (e) PDE loss during training. Once the values of both the adaptive constants start dropping, the PDE loss improves much more rapidly. (f) PDE loss without weight decay. The loss stagnates at a relatively higher value.

Section 4.3 uses a hybrid data–PINN approach to generate a surrogate model for a given design space of a heat sink with a chip underneath it, undergoing cooling via forced convection.

Then, given certain constraints on the running metrics of the chip-sink setup (like max temperature in the chip), the optimal set of parameters in the DoE space that satisfy the constraints while maximizing an objective are obtained via rapid design optimization using the created surrogate. The experiment shows the effectiveness of the PINN surrogate in modeling the flow and convective heat transfer of the problem.

Details on hyperparameters used in the model training for each experiment that follows can be found in Appendix A.1.

4.1 Forward Solve of 3D Stenosis Problem

Flow through an idealized 3D stenosis geometry at a physiologically relevant Reynolds number is demonstrated; see Fig. 3 for details about the geometry. To the author's best knowledge, flow through a stenosis has been solved using PINNs only at a low Reynolds number of approximately 6 (based on inlet diameter) (Sun et al., 2020). Flow through irregular geometries has been solved at a higher Re (500) but in 2D format (Oldenburg et al., 2022). In this paper, the stenosis problem is solved at Re 150 and in three dimensions.

As discussed in Section 2.2, at higher Reynolds numbers, the standard PINN implementation struggles to achieve a good local minimum. This was confirmed using a standard PINN implementation. To alleviate this issue, a data-assisted approach was taken, where sporadic solution data can be added throughout the domain of interest (depicted on a slice in Fig. 4).

4.1.1 Problem Setup

The flow problem through the stenosis is solved by solving the steady-state incompressible Navier–Stokes equations:

$$\nabla \cdot \mathbf{u} = 0, \tag{9}$$

$$(\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla \mathbf{p} + \nu \nabla \cdot (\nabla \mathbf{u}), \tag{10}$$

subject to

$$\mathbf{u}(x_{b1}) = g(x_{b1}), \quad x_{b1} \in \partial\Omega_1,$$
$$\mathbf{u}(x_{b2}) = 0, \quad x_{b2} \in \partial\Omega_2,$$

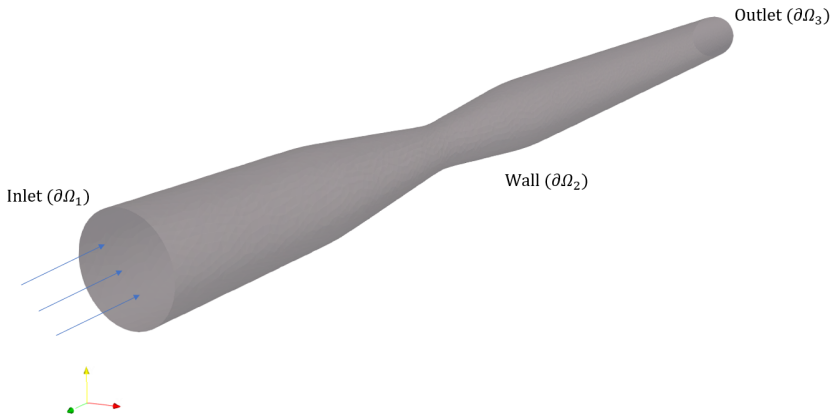


FIG. 3: Visual description of stenosis problem

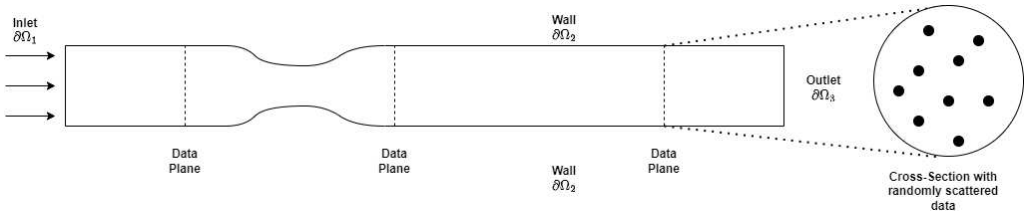


FIG. 4: Stenosis diagram (not to scale) showing planes where solution data is provided randomly

$$\nabla u_i(x_{b3}) \cdot \mathbf{n} = 0, \quad x_{b3} \in \partial\Omega_3, \quad i = 1, 2, 3,$$
$$p(x_{b3}) = 0, \quad x_{b3} \in \partial\Omega_3,$$

where $g(x_{b3})$ represents a profiled input to the stenosis. ρ and ν are the density and kinematic viscosity of the fluid (air), respectively, and $\mathbf{u}$ and p are the velocity vector and pressure, respectively.

In the present problem, a parabolic profiled input is provided with a peak value inflow of 0.15 m/s. The ratio of areas of the throat to the inlet is 0.36.

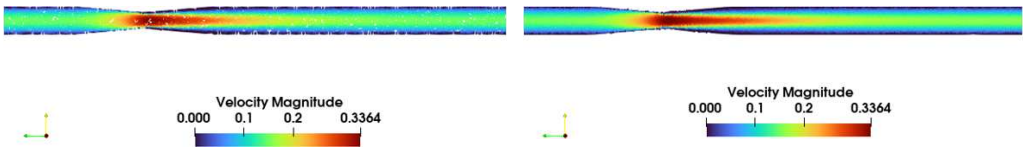
The output of the network is approximated as G_θ, which is a four-component output:

$$G_\theta = \begin{bmatrix} u \\ v \\ w \\ p \end{bmatrix}.$$

4.1.2 Results

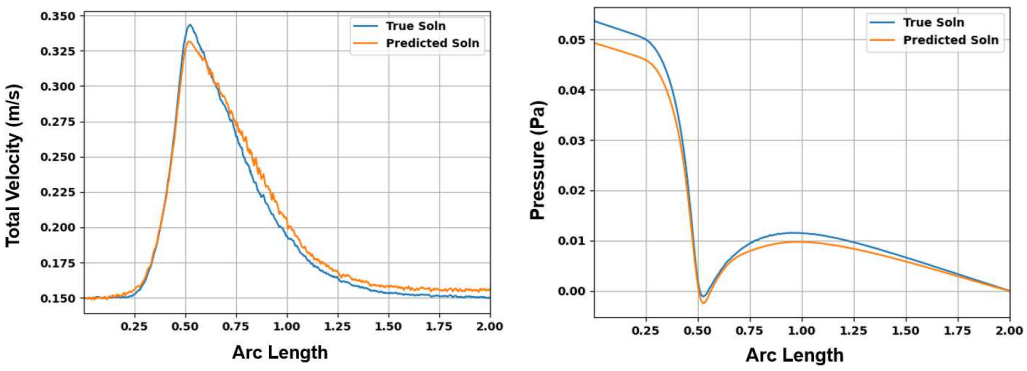
Figure 5 compares the velocity magnitude returned by the trained PINN model and Altair AcuSolve® through a 2D slice of the stenosis. As can be seen, the essential features of the flow are captured. Figure 6 compares the velocity and pressure profile through the center of the stenosis. The differences between the line plots are attributed to differences in mesh density between the two cases. The CFD mesh was an unstructured mesh of around 63,000 nodes with a boundary layer, while the point cloud used with the PINN was around 87,000 nodes, randomly distributed points except near the boundary, where the sampling was finer. Without the use of sparse domain data, the trained PINN simply predicts the trivial solution, i.e., $\mathbf{u} = 0 \ \ \forall x \in \Omega - \partial\Omega_1$.

Another approach that was investigated to solve the 3D stenosis problem was that of using "continuity planes" as defined by Hennigh et al. (2021) in their experiments solving 3D flow problems using PINNs. In this approach, the authors added constraints on the mass flow through



(a) (b)

FIG. 5: Solution comparison: (a) Altair AcuSolve® solution to stenosis problem; (b) PINN forward solve to stenosis problem



(a) (b)

FIG. 6: Centerline solution comparisons: PINN versus Altair AcuSolve®; (a) total velocity comparison, (b) pressure comparison

a plane and added these constraints to the loss function. While this approach was found to aid the convergence of the PINN model to the correct solution, there were several issues found to exist with this method:

1. It is difficult to generate continuity planes for complex geometries such as those shown in Sections 4.2 and 4.3.

2. The quality of the solution from the PINN depends heavily on the integration scheme used to calculate the mass flow rate and the fineness of the points on the continuity plane.

Hence, in the next section, random and sparsely distributed data was used in the domain to aid convergence.

4.2 Flow over a Printed Circuit Board (PCB)

4.2.1 Problem Setup

Flow over a PCB consisting of a heat sink, chip, and capacitor is solved at a Reynolds number of approximately 1500, based on the length of the PCB board and air as the fluid. The geometry and flow orientation are shown in Fig. 7. This represents a forced convection problem common in electronics design and is a challenging problem for PINNs because it is 3D, with complex geometry and large gradients in the solution variables involved.

Let D represent the set of all nodes in the domain. To train the PINN model, the CFD solution was first computed. Next, 1% of the nodes in the solution domain were randomly selected (call this set $D_1 \subset D$). This is a selection of roughly 2,300 node points (from a mesh of roughly 230,000 nodes). The experiment was then divided into three parts:

1. **Case A:** A network was trained on the CFD solution at all points in D_1 (i.e., $\forall \mathbf{x} \in D_1$), following which the physics of the problem was enforced at every node location in D (i.e., $\forall \mathbf{x} \in D$) by including the physics-based loss in the training, and then the network was asked to predict the solution in the entire domain D.

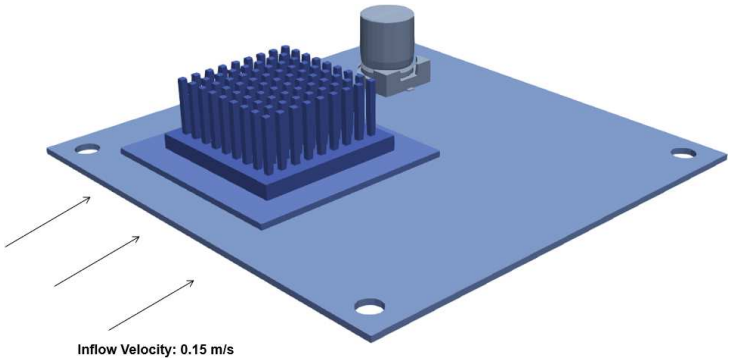


FIG. 7: Geometry of a PCB with a chip, sink, and capacitor assembly

2. **Case B:** A network was trained on the CFD solution at the points contained in D_1 (i.e., $\forall \mathbf{x} \in D_1$) without any physics enforcement and then asked to predict the solution in the entire domain (i.e., $\forall \mathbf{x} \in D$).

3. **Case C:** Finally, the same experiment as Case A was repeated but with a new set (D_2) consisting of only 0.2% of the nodes in D, which were again randomly selected.

The governing equations for this problem are the Reynolds-averaged Navier–Stokes equations:

$$\nabla \cdot \mathbf{u} = 0, \tag{11}$$

$$(\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla \mathbf{p} + (\nu + \nu_t)\nabla \cdot (\nabla \mathbf{u}). \tag{12}$$

ρ, ν, and ν_t represent the density, kinematic viscosity, and eddy viscosity of the system. The inflow is set to a constant velocity of 0.15 m/s, and the outflow is set to the stress-free condition. It should be noted that in the current study, eddy viscosity is obtained directly from the CFD solver using the Spalart–Allmaras turbulence model (Spalart and Allmaras, 1992). Turbulence modeling in PINNs is a field of active research with a few articles investigating it (Eivazi et al., 2022; Ghosh et al., 2023; Hennigh et al., 2021), and it is a future goal to effectively incorporate turbulence models into PINN-based models.

4.2.2 Results

Figure 8 shows the ANN predictions for the different cases. It is evident that by using sparse data, the network is able to better converge toward the CFD solution [shown in Fig. 8(d)] using the physics-based regularizer. However, as evident in Fig. 8(c), the network failed to converge to a physical solution when the amount of data provided was insufficient, highlighting the importance of a certain amount and fineness of the required data. Table 1 shows the mean squared errors (MSE) for each experiment, for the velocity and the pressure, taking the CFD solution as the ground truth. The MSE is calculated as

$$\text{MSE} = \sqrt{\frac{\sum_{i=1}^{N_{\text{nodes}}} (x_{i,\text{pred}} - x_{i,\text{truth}})^2}{N_{\text{nodes}}}}. \tag{13}$$

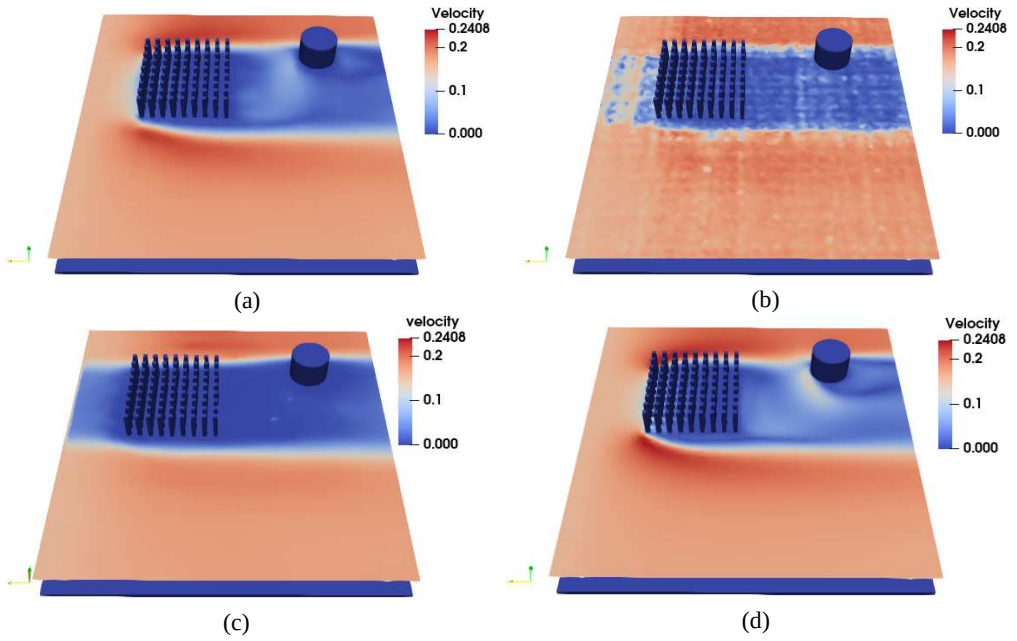


FIG. 8: Neural network (NN) prediction with and without physics for very coarse data supplied on a plane through the domain. (a) **Case A:** trained on 1% data and physics; (b) **Case B:** trained on 1% solution data only; (c) **Case C:** trained on 0.2% data and physics; (d) True Solution from CFD solver.

TABLE 1: MSE for velocity and pressure for cases A, B, and C

Case	Description	MSE (Velocity)	MSE (Pressure)
Case A	1% domain data + physics	**0.0135**	**0.0037**
Case B	1% domain data only	0.0222	0.00472
Case C	0.2% domain data + physics	0.0245	0.00545

Figure 9 shows the pointwise magnitude of the errors in the total velocity. In each case, the errors are concentrated near the boundary of the sink, but the presence of adequate data and the physics reduces this the most. Figure 10 shows the probability density function (PDF) for the absolute error in velocity magnitude and pressure. The errors in Case A are much lower than those of Case B and Case C for both predicted velocity magnitude and pressure. These results open exciting possibilities for using physics-based regularizers in the future and represent a step forward for solving the 3D Navier–Stokes equations at high Reynolds numbers using PINNs. Furthermore, data generation costs to create surrogate models using PINNs can be greatly reduced by providing solution data on a coarser grid and solving the physics on a finer grid.

4.3 Surrogate Modeling and Design Optimization of a Heat Sink

In this section, PINNs are used to create a surrogate model of a heat sink assembly undergoing forced convection. The assembly utilizes a chip that generates heat and a fin-type heatsink on top to dissipate heat into the surrounding fluid and is cooled by forced convection of air. The geometry and setup are shown in Fig. 11.

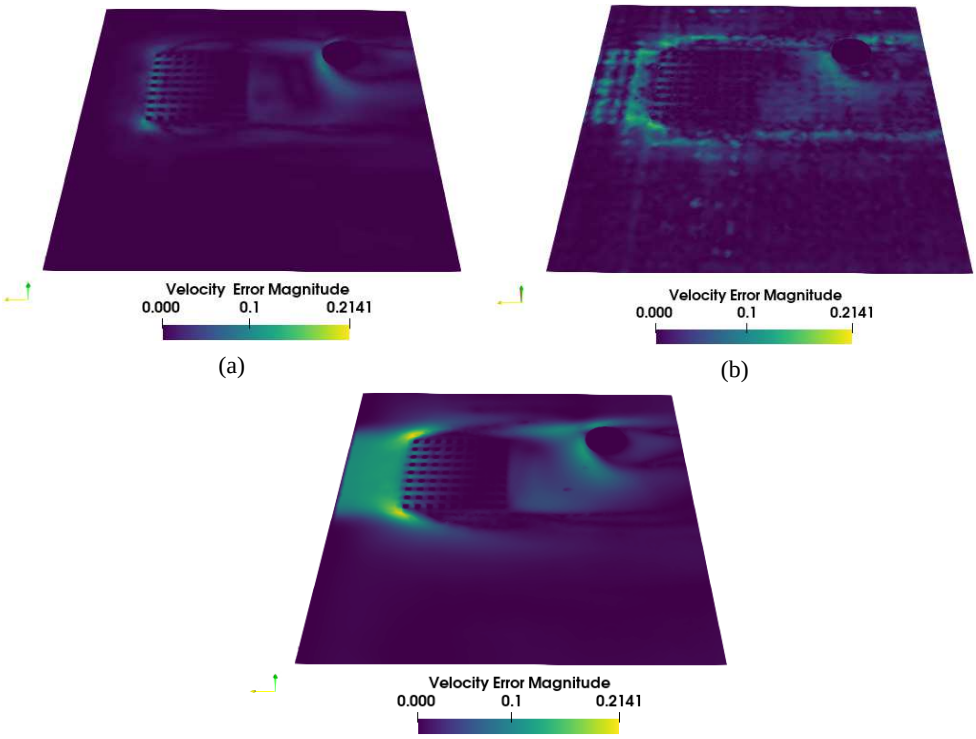


FIG. 9: Pointwise magnitude of error in total velocity. (a) **Case A:** trained on 1% data and physics; (b) **Case B:** trained on 1% solution data only; (c) **Case C:** trained on 0.2% data and physics.

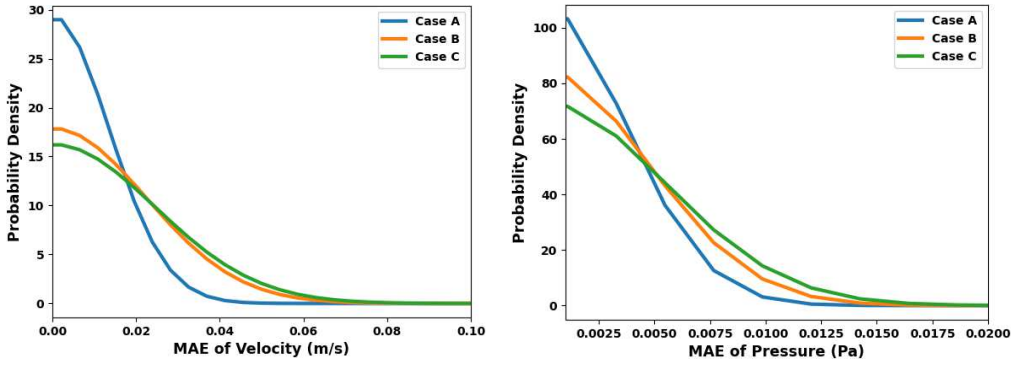


FIG. 10: Probability density functions (PDFs) of errors for each case described: (a) error PDF of velocity, (b) error PDF of pressure

To show the effectiveness of PINNs in creating accurate surrogate models for the setup, two experiments are conducted. Section 4.3.1 describes the model creation method and tabulates the mean error in the predicted full-field flow thermal solution at a few test points in the design domain of interest. These errors are also compared to the errors of the prediction by a standard

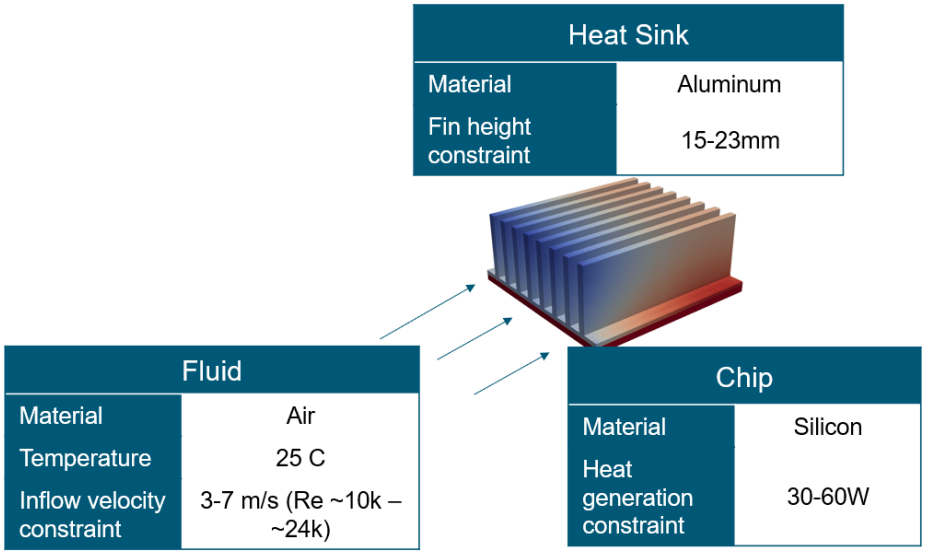


FIG. 11: Basic problem geometry and flow depiction

data-driven NN, identically constructed in every way to the PINN except that it does not leverage the key features discussed in Section 3.

Then, in Section 4.3.2, the PINN surrogate is used to solve a design optimization problem on the heat sink assembly. The goal is to optimize the heat sink design and the running conditions of the assembly, subject to feasibility constraints placed on the chip temperature and channel pressure drop. The ability of the surrogate to return the optimal solution of the design problem further highlights the capabilities of the model to accurately predict the full field solution in the entire DoE space.

The technical details of the design problem are as follows: if $\dot{Q}_{\text{src}}$ is the total power being generated by the chip, the optimization problem can be framed as:

$$\text{Maximize } \dot{Q}_{\text{src}} \text{ s.t.}, \tag{14}$$

$$\text{Pressure drop across the heat sink channel } (\Delta P) \leq 11 \text{ Pa}, \tag{15}$$

$$\text{Maximum temperature anywhere on the chip} \leq 350 \text{ K}. \tag{16}$$

The pressure at the outflow is fixed to 0 Pa, and the pressure drop across the heat sink channel is hence calculated as the average pressure over the inflow of the channel:

$$\Delta P = \overline{P_{\text{inlet}}}. \tag{17}$$

The term to be maximized $\dot{Q}_{\text{src}}$ is also one of the design axes and an input parameter (P3) to the network.

The design variables that can be altered for this present optimization are

- Inflow velocity
- Fin height
- Source term in the chip (has to be maximized)

The upper and lower limits of each of the design variables mentioned above are summarized in Table 2. The inlet velocity is set based on typical values found in literature (Lindstedt and Karvinen, 2012) and corresponds to a Reynolds number range of Re 10,300 to 24,000.

The governing equations solved for this conjugate heat transfer problem are the same as in Section 4.2 for the flow problem, subject to no-slip boundary conditions on the chip heatsink assembly with a variable freestream inflow velocity, causing forced convection. As in Section 4.2, the eddy viscosities are taken from the CFD solutions.

The energy equation in both fluid and solid reads as follows:

$$k\nabla^2 T + \dot{q}_{\text{src}} - \rho s \mathbf{u} \cdot \nabla T = 0, \tag{18}$$

where T represents the temperature, $\dot{q}_{\text{src}}$ represents the volumetric source term, and k and s are the conductivity and specific heat of the material, respectively. At the interface between the fluid and solid domain (fluid-sink, sink-chip, and fluid-chip), the interface condition is applied by minimizing the following loss terms, as shown in Jagtap et al. (2020):

$$L_{\text{flux}} = \frac{1}{N_{\text{int}}} \sum_{i=1}^{N_{\text{int}}} \left(f_{d_1}(\mathbf{u}(x_i)) \cdot \mathbf{n}_{d_1} + f_{d_2}(\mathbf{u}(x_i)) \cdot \mathbf{n}_{d_2} \right)^2, \tag{19}$$

$$L_{\text{val}} = \frac{1}{N_{\text{int}}} \sum_{i=1}^{N_{\text{int}}} \left(\mathbf{u}_{d_j}(x_i) - \overline{\mathbf{u}_{d_j}(x_i)} \right)^2, \tag{20}$$

where $\mathbf{n}_{d1} = -\mathbf{n}_{d2}$, and $j = 1, 2$. The average is taken over j. d_1 and d_2 refer to the domains on both sides of the interface, and N_{int} is the number of node points on the interface.

4.3.1 Model Creation and Evaluation

The sampling of the above DoE space is done via an efficient space-sampling method to optimally fill the DoE space, described thoroughly in Appendix A.3. The sampled DoE space for training is shown in Fig. 12, along with the location of points at which the surrogate model is tested. The reader is referred to Appendix A.4.1 for a complete tabular description of the DoE space. Note that for this example, we use full field data at each DoE point to train the surrogate, as opposed to a small fraction of it (as in Sections 4.1 and 4.2), as the objective is to get a surrogate that is as accurate as possible. Table 3 shows the MSE for the predictions by the hybrid data–PINN model and the standard data-driven NN model at the test points, calculated using the CFD solution at the same mesh resolution. The hybrid data–PINN model outperforms the standard data-driven NN for all predictions.

TABLE 2: Design of experiments space axes ranges for the heat sink design optimization

Parameter No.	Parameter name	Lower value	Upper value
P1	inflow velocity (m/s)	3	7
P2	fin height (mm)	15	23
P3	source term (W)	30	60

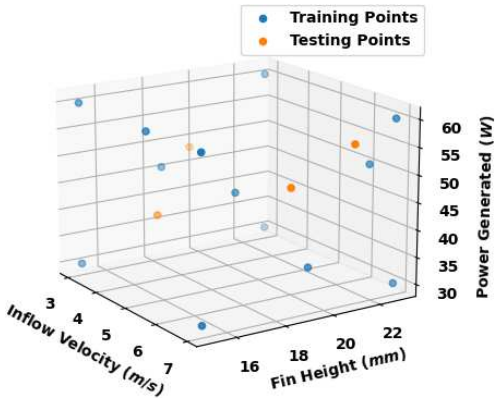


FIG. 12: Training and testing points in the 3D DoE space

TABLE 3: MSE for four test points shown in Table A2. The PINN-based model consistently outperforms the standard data-driven NN on all test points

	Velocity MSE	Pressure MSE	Temperature MSE
Test point 1			
Hybrid data–PINN model	**0.205**	**0.862**	**1.031**
Standard data-driven NN	0.711	1.387	1.391
Test point 2			
Hybrid data–PINN model	**0.130**	**0.371**	**1.499**
Standard data-driven NN	0.449	0.860	1.933
Test point 3			
Hybrid data–PINN model	**0.206**	**1.193**	**0.782**
Standard data-driven NN	0.835	1.660	1.374
Test point 4			
Hybrid data–PINN model	**0.101**	**0.297**	**1.261**
Standard data-driven NN	0.417	0.725	1.838

4.3.2 Solving the Design Optimization Problem

The PINN surrogate model is used to solve the design optimization problem described in Eqs. (14)–(16). The created surrogate models are interfaced with an optimizer that solves a generic constrained optimization problem via an iterative process, described thoroughly in Appendix A.2.

Each snapshot in Fig. 13 represents a design iteration, and each particle represents a point in the DoE space. Each axis of a plot represents a parameter axis.

For the given constraints, the particles converge to a much smaller region of the DoE space. The design point returned by the optimizer in this case is

Inflow velocity: 6 m/s
Chip power: 50 W
Fin height: 17 mm

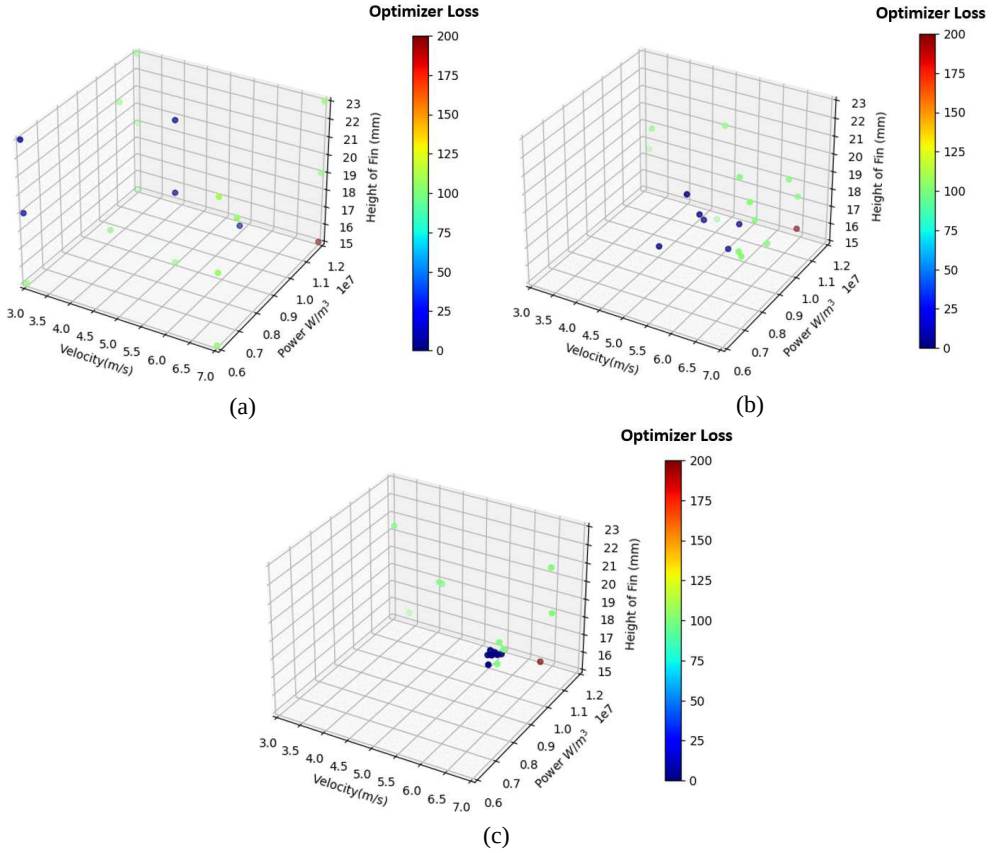


FIG. 13: Design optimization iterations of the heat sink problem: (a) Iteration 0, (b) Iteration 5, (c) Iteration 10

To test that the result satisfies the constraints, the returned design point is solved by the Altair AcuSolve® at the same mesh fineness and another mesh with 10× fineness, keeping all essential mesh features such as boundary layers and refinement zones. As shown in Figs. 14 and 15, not only does the given design point satisfy the design constraints, but the finer mesh solution is extremely close to the coarser solution, and a little tweaking of the design point using CFD with the higher-resolution mesh will yield a highly optimized solution to the designer. This optimization is done several orders of magnitude faster than if using traditional CFD, and the reader is referred to Appendix A.4.2 for a quantitative description of the same.

5. CONCLUSIONS AND FUTURE WORK

In this paper, PINNs were used to solve the 3D Navier–Stokes equations in a data-assisted setting for complex geometries with realistic physical parameters. It was shown that even for problems being solved at high Reynolds numbers in 3D, PINNs can be trained to produce a good solution in the presence of sparse solution data randomly scattered in the solution domain. However, using too little solution data causes the model to converge to an unphysical solution. PINNs were also demonstrated for 3D flow thermal surrogate modeling, and the PINN-based surrogates

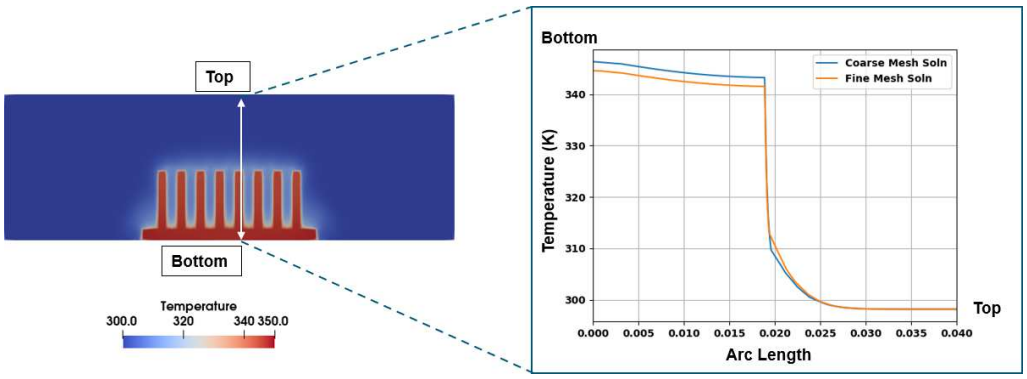


FIG. 14: Temperature plot through a slice at the rear of the sink (from bottom to top). The comparison between the high-fidelity solution on the fine mesh and the PINN prediction on a coarser mesh shows good agreement.

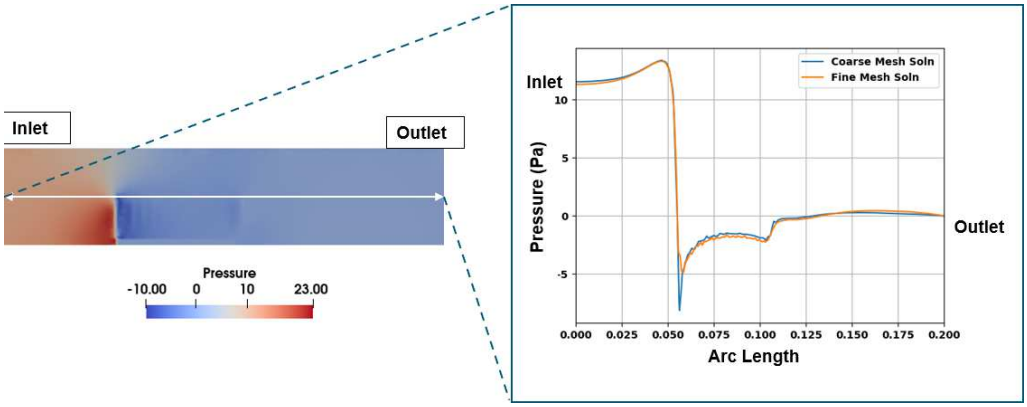


FIG. 15: Pressure plot through a slice through the middle of the flow channel (from left to right). The comparison between the high-fidelity solution on the fine mesh and the PINN prediction on a coarser mesh shows good agreement.

consistently outperformed standard data-driven NN on test case examples. The PINN surrogates were also interfaced with a design optimization algorithm to solve a constrained optimization problem. This optimization returned a design point that when solved with high-fidelity CFD was consistent with the requirements of the design constraints, highlighting the suitability of the method to produce surrogates for 3D flow thermal surrogate modeling problems.

There are multiple avenues through which the work shown in this paper can be improved. Research has to be done to improve convergence and offer guarantees of PINN training toward the local minimums that represent physical solutions in a data-free manner. This will further reduce data requirements for the creation of physically consistent PINN models, which can greatly improve their surrogate modeling capabilities by reducing the cost of training and improving predictive accuracy. Further work needs to be done to investigate turbulence modeling in PINNs so that high Reynolds number problems can be solved in a data-free manner. There are also many themes such as uncertainty quantification (Bharadwaja et al., 2022; Yang et al., 2021; Zhang et al., 2019) of surrogates and effective surrogate modeling of different geometries (Gao et al., 2021;

Kashefi et al., 2021; Oldenburg et al., 2022) that are active fields of research in PINNs, which could be included in future works that build on these results.

ACKNOWLEDGMENTS

This research did not receive any specific grant from funding agencies in the public or not-for-profit sectors, or from any external commercial entities. The authors gratefully acknowledge the use of Altair Engineering Inc.'s computing facilities for running experiments.

AUTHORSHIP CONTRIBUTION STATEMENT

Saakaar Bhatnagar: formal analysis, investigation, methodology, software, validation, writing—original draft. **Andrew Comerford:** conceptualization, investigation, project administration, supervision, writing—review and editing. **Araz Banaeizadeh:** conceptualization, project administration, supervision, writing—review and editing.

REFERENCES

Afzal, A., Kim, K.Y., and Seo, J.W., Effects of Latin Hypercube Sampling on Surrogate Modeling and Optimization, *Int. J. Fluid Mach. Syst.*, vol. **10**, no. 3, pp. 240–253, 2017.

Antonello, F., Buongiorno, J., and Zio, E., Physics Informed Neural Networks for Surrogate Modeling of Accidental Scenarios in Nuclear Power Plants, *Nucl. Eng. Technol.*, vol. **55**, no. 9, pp. 3409–3416, 2023.

Basir, S. and Senocak, I., Critical Investigation of Failure Modes in Physics-Informed Neural Networks, *AIAA SCITECH 2022 Forum*, p. 2353, 2022.

Bharadwaja, B., Nabian, M.A., Sharma, B., Choudhry, S., and Alankar, A., Physics-Informed Machine Learning and Uncertainty Quantification for Mechanics of Heterogeneous Materials, *Integrat. Mater. Manuf. Innov.*, vol. **11**, no. 4, pp. 607–627, 2022.

Bhatnagar, S., Afshar, Y., Pan, S., Duraisamy, K., and Kaushik, S., Prediction of Aerodynamic Flow Fields Using Convolutional Neural Networks, *Comput. Mech.*, vol. **64**, pp. 525–545, 2019.

Bischof, R. and Kraus, M., Multi-Objective Loss Balancing for Physics-Informed Deep Learning, arXiv preprint arXiv:2110.09813, 2021.

Brunton, S.L., Noack, B.R., and Koumoutsakos, P., Machine Learning for Fluid Mechanics, *Annu. Rev. Fluid Mech.*, vol. **52**, pp. 477–508, 2020.

Cai, S., Mao, Z., Wang, Z., Yin, M., and Karniadakis, G.E., Physics-Informed Neural Networks (PINNs) for Fluid Mechanics: A Review, *Acta Mechanica Sinica*, vol. **37**, no. 12, pp. 1727–1738, 2021a.

Cai, S., Wang, Z., Fuest, F., Jeon, Y.J., Gray, C., and Karniadakis, G.E., Flow over an Espresso Cup: Inferring 3-D Velocity and Pressure Fields from Tomographic Background Oriented Schlieren via Physics-Informed Neural Networks, *J. Fluid Mech.*, vol. **915**, p. A102, 2021b.

Cai, S., Wang, Z., Wang, S., Perdikaris, P., and Karniadakis, G.E., Physics-Informed Neural Networks for Heat Transfer Problems, *J. Heat Transf.*, vol. **143**, no. 6, p. 060801, 2021c.

Calzolari, G. and Liu, W., Deep Learning to Replace, Improve, or Aid CFD Analysis in Built Environment Applications: A Review, *Build. Environ.*, vol. **206**, p. 108315, 2021.

Chen, Y., Lu, L., Karniadakis, G.E., and Dal Negro, L., Physics-Informed Neural Networks for Inverse Problems in Nano-Optics and Metamaterials, *Opt. Express*, vol. **28**, no. 8, pp. 11618–11633, 2020.

Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., and Piccialli, F., Scientific Machine Learning through Physics-Informed Neural Networks: Where We Are and What's Next, *J. Sci. Comput.*, vol. **92**, no. 3, p. 88, 2022.

Das, S. and Tesfamariam, S., State-of-the-Art Review of Design of Experiments for Physics-Informed Deep Learning, arXiv preprint arXiv:2202.06416, 2022.

Eivazi, H., Tahani, M., Schlatter, P., and Vinuesa, R., Physics-Informed Neural Networks for Solving Reynolds-Averaged Navier–Stokes Equations, *Phys. Fluids*, vol. **34**, no. 7, p. 075117, 2022.

Finn, C., Abbeel, P., and Levine, S., Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks, *Int. Conf. on Machine Learning*, Sydney, Australia, pp. 1126–1135, 2017.

Gao, H., Sun, L., and Wang, J.X., PhyGeoNet: Physics-Informed Geometry-Adaptive Convolutional Neural Networks for Solving Parameterized Steady-State PDEs on Irregular Domain, *J. Comput. Phys.*, vol. **428**, p. 110079, 2021.

Ghosh, S., Chakraborty, A., Brikis, G.O., and Dey, B., RANS-PINN Based Simulation Surrogates for Predicting Turbulent Flows, arXiv preprint arXiv:2306.06034, 2023.

Gladstone, R.J., Nabian, M.A., and Meidani, H., FO-PINNs: A First-Order Formulation for Physics Informed Neural Networks, arXiv preprint arXiv:2210.14320, 2022.

Gopakumar, V., Pamela, S., and Samaddar, D., Loss Landscape Engineering via Data Regulation on PINNs, *Mach. Learn. Appl.*, vol. **12**, p. 100464, 2023.

Guo, X., Li, W., and Iorio, F., Convolutional Neural Networks for Steady Flow Approximation, *Assoc. Comput. Mach.*, pp. 481–490, 2016.

Hansen, D., Maddix, D.C., Alizadeh, S., Gupta, G., and Mahoney, M.W., Learning Physical Models That Can Respect Conservation Laws, *Physica D*, vol. **457**, p. 133952, 2024.

He, H. and Pathak, J., An Unsupervised Learning Approach to Solving Heat Equations on Chip Based on Auto-Encoder and Image Gradient, arXiv preprint arXiv:2007.09684, 2020.

Hennigh, O., Narasimhan, S., Nabian, M.A., Subramaniam, A., Tangsali, K., Fang, Z., Rietmann, M., Byeon, W., and Choudhry, S., NVIDIA SimNet™: An AI-Accelerated Multi-Physics Simulation Framework, *Int. Conf. on Computational Science*, Kraków, Poland, pp. 447–461, 2021.

Hu, Z., Jagtap, A.D., Karniadakis, G.E., and Kawaguchi, K., When Do Extended Physics-Informed Neural Networks (XPINNS) Improve Generalization?, *SIAM J. Sci. Comput.*, vol. **44**, no. 5, pp. A3158–A3182, 2022.

Ishitsuka, K. and Lin, W., Physics-Informed Neural Network for Inverse Modeling of Natural-State Geothermal Systems, *Appl. Energy*, vol. **337**, p. 120855, 2023.

Jagtap, A.D., Kharazmi, E., and Karniadakis, G.E., Conservative Physics-Informed Neural Networks on Discrete Domains for Conservation Laws: Applications to Forward and Inverse Problems, *Comput. Methods Appl. Mech. Eng.*, vol. **365**, p. 113028, 2020.

Jagtap, A.D., Mao, Z., Adams, N., and Karniadakis, G.E., Physics-Informed Neural Networks for Inverse Problems in Supersonic Flows, *J. Comput. Phys.*, vol. **466**, p. 111402, 2022.

Kashefi, A., Rempe, D., and Guibas, L.J., A Point-Cloud Deep Learning Framework for Prediction of Fluid Flow Fields on Irregular Geometries, *Phys. Fluids*, vol. **33**, no. 2, p. 027104, 2021.

Kennedy, J. and Eberhart, R., Particle Swarm Optimization, *Proc. of ICNN'95 – Int. Conf. on Neural Networks*, Perth, Australia, Vol. 4, pp. 1942–1948, 1995.

Kim, S.W., Kwak, E., Kim, J.H., Oh, K.Y., and Lee, S., Modeling and Prediction of Lithium-Ion Battery Thermal Runway via Multiphysics-Informed Neural Network, *J. Energy Storage*, vol. **60**, p. 106654, 2023.

Kingma, D.P. and Ba, J., Adam: A Method for Stochastic Optimization, arXiv preprint arXiv:1412.6980, 2014.

Krath, E.H., Carpenter, F.L., Cizmas, P.G., and Johnston, D.A., An Efficient Proper Orthogonal Decomposition Based Reduced-Order Model for Compressible Flows, *J. Comput. Phys.*, vol. **426**, p. 109959, 2021.

Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., and Mahoney, M.W., Characterizing Possible Failure Modes in Physics-Informed Neural Networks, *Adv. Neural Inf. Process. Syst.*, vol. **34**, pp. 26548–26560, 2021.

Lee, S. and You, D., Data-Driven Prediction of Unsteady Flow over a Circular Cylinder Using Deep Learning, *J. Fluid Mech.*, vol. **879**, pp. 217–254, 2019.

Li, Z., Kovachki, N.B., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A.M., and Anandkumar, A., Fourier Neural Operator for Parametric Partial Differential Equations, arXiv preprint arXiv:2010.08895, 2020.

Lin, D.K., Simpson, T.W., and Chen, W., Sampling Strategies for Computer Experiments: Design and Analysis, *Int. J. Reliab. Appl.*, vol. **2**, no. 3, pp. 209–240, 2001.

Lindstedt, M. and Karvinen, R., Optimization of Plate Fin Arrays with Laminar and Turbulent Forced Convection, *J. Phys.: Conf. Ser.*, vol. **395**, p. 012059, 2012.

Liu, R.L., Hua, Y., Zhou, Z.F., Li, Y., Wu, W.T., and Aubry, N., Prediction and Optimization of Airfoil Aerodynamic Performance Using Deep Neural Network Coupled Bayesian Method, *Phys. Fluids*, vol. **34**, no. 11, p. 117116, 2022.

Liu, Z., Yang, Y., and Cai, Q.D., Solving Differential Equation with Constrained Multilayer Feedforward Network, arXiv preprint arXiv:1904.06619, 2019.

Melas-Kyriazi, L., The Mathematical Foundations of Manifold Learning, arXiv preprint arXiv:2011.01307, 2020.

Morris, M.D. and Mitchell, T.J., Exploratory Designs for Computational Experiments, *J. Stat. Plann. Inference*, vol. **43**, no. 3, pp. 381–402, 1995.

Nekkanti, A. and Schmidt, O.T., Gappy Spectral Proper Orthogonal Decomposition, *J. Comput. Phys.*, vol. **478**, p. 111950, 2023.

Oldenburg, J., Borowski, F., Öner, A., Schmitz, K.P., and Stiehm, M., Geometry Aware Physics Informed Neural Network Surrogate for Solving Navier–Stokes Equation (GAPINN), *Adv. Model. Simul. Eng. Sci.*, vol. **9**, no. 1, p. 8, 2022.

Penwarden, M., Zhe, S., Narayan, A., and Kirby, R.M., A Metalearning Approach for Physics-Informed Neural Networks (PINNs): Application to Parameterized PDEs, *J. Comput. Phys.*, vol. **477**, p. 111912, 2023.

Raissi, M. and Karniadakis, G.E., Hidden Physics Models: Machine Learning of Nonlinear Partial Differential Equations, *J. Comput. Phys.*, vol. **357**, pp. 125–141, 2018.

Raissi, M., Perdikaris, P., and Karniadakis, G., Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations, *J. Comput. Phys.*, vol. **378**, pp. 686–707, 2019.

Rao, C., Sun, H., and Liu, Y., Physics-Informed Deep Learning for Computational Elastodynamics without Labeled Data, *J. Eng. Mech.*, vol. **147**, no. 8, p. 04021043, 2021.

Saad, N., Gupta, G., Alizadeh, S., and Robinson, D.M., Guiding Continuous Operator Learning through Physics-Based Boundary Constraints, *ICLR 2023*, 2023.

Sharma, R., Raissi, M., and Guo, Y., Physics-Informed Deep Learning of Gas Flow-Melt Pool Multi-Physical Dynamics during Powder Bed Fusion, *CIRP Annals*, 2023.

Sitzmann, V., Martel, J., Bergman, A., Lindell, D., and Wetzstein, G., Implicit Neural Representations with Periodic Activation Functions, *Adv. Neural Inf. Proc. Syst.*, vol. **33**, pp. 7462–7473, 2020.

Son, H., Cho, S.W., and Hwang, H.J., Enhanced Physics-Informed Neural Networks with Augmented Lagrangian Relaxation Method (AL-PINNs), *Neurocomputing*, p. 126424, 2023.

Spalart, P. and Allmaras, S., A One-Equation Turbulence Model for Aerodynamic Flows, *30th Aerospace Sciences Meeting and Exhibit*, Reno, NV, p. 439, 1992.

Subramanian, S., Kirby, R.M., Mahoney, M.W., and Gholami, A., Adaptive Self-Supervision Algorithms for Physics-Informed Neural Networks, arXiv preprint arXiv:2207.04084, 2022.

Sukumar, N. and Srivastava, A., Exact Imposition of Boundary Conditions with Distance Functions in Physics-Informed Deep Neural Networks, *Comput. Methods Appl. Mech. Eng.*, vol. **389**, p. 114333, 2022.

Sun, L., Gao, H., Pan, S., and Wang, J.X., Surrogate Modeling for Fluid Flows Based on Physics-Constrained Deep Learning without Simulation Data, *Comput. Methods Appl. Mech. Eng.*, vol. **361**, p. 112732, 2020.

Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., and Ng, R., Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains, *Adv. Neural Inf. Process. Syst.*, vol. **33**, pp. 7537–7547, 2020.

Wang, K., Chen, Y., Mehana, M., Lubbers, N., Bennett, K.C., Kang, Q., Viswanathan, H.S., and Germann, T.C., A Physics-Informed and Hierarchically Regularized Data-Driven Model for Predicting Fluid Flow through Porous Media, *J. Comput. Phys.*, vol. **443**, p. 110526, 2021a.

Wang, S., Teng, Y., and Perdikaris, P., Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks, *SIAM J. Sci. Comput.*, vol. **43**, no. 5, pp. A3055–A3081, 2021b.

Wang, S., Wang, H., and Perdikaris, P., On the Eigenvector Bias of Fourier Feature Networks: From Regression to Solving Multi-Scale PDEs with Physics-Informed Neural Networks, *Comput. Methods Appl. Mech. Eng.*, vol. **384**, p. 113938, 2021c.

Wang, S., Wang, H., and Perdikaris, P., Improved Architectures and Training Algorithms for Deep Operator Networks, *J. Sci. Comput.*, vol. **92**, no. 2, p. 35, 2022a.

Wang, S., Yu, X., and Perdikaris, P., When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective, *J. Comput. Phys.*, vol. **449**, p. 110768, 2022b.

Wu, C., Zhu, M., Tan, Q., Kartha, Y., and Lu, L., A Comprehensive Study of Non-Adaptive and Residual-Based Adaptive Sampling for Physics-Informed Neural Networks, *Comput. Methods Appl. Mech. Eng.*, vol. **403**, p. 115671, 2023.

Yang, L., Meng, X., and Karniadakis, G.E., B-PINNs: Bayesian Physics-Informed Neural Networks for Forward and Inverse PDE Problems with Noisy Data, *J. Comput. Phys.*, vol. **425**, p. 109913, 2021.

Zhang, D., Lu, L., Guo, L., and Karniadakis, G.E., Quantifying Total Uncertainty in Physics-Informed Neural Networks for Solving Forward and Inverse Stochastic Problems, *J. Comput. Phys.*, vol. **397**, p. 108850, 2019.

Zhao, Y., Akolekar, H.D., Weatheritt, J., Michelassi, V., and Sandberg, R.D., RANS Turbulence Model Development Using CFD-Driven Machine Learning, *J. Comput. Phys.*, vol. **411**, p. 109413, 2020.

APPENDIX A.

A.1 PINN Model Architecture and Training Details

Every PINN model trained was created using fully connected layers. There were two sizes of models used:

1. For models simulating a flow (as in Sections 4.1, 4.2, and 4.3), a network three layers deep with 128 hidden units per layer was used.

2. For models simulating the temperature in a body (as in Section 4.3), a network two layers deep with 64 hidden units was used for each body.

This distinction was made because

1. Fluid domains tend to have more nodes due to being larger and having special mesh structures such as boundary layers.
2. The solution in the flow domain tends to be more complex and hence needs more representation capacity.
3. For thermal solves we use a domain decomposition strategy (described below) so that the network representing each subdomain itself can be smaller, which leads to faster training.

For more information on the multi-model domain decomposition approach, see Appendix A.1.1.

The optimizer used was the Adam (Kingma and Ba, 2014) optimizer, and the training points and data were divided into mini batches of a randomly selected subset of the total training points, usually of size 1%–5% of the total available points. These mini batches were re-sampled every 5000 training epochs. The initial LR was set to be 10^{-3} with a decay factor of 0.9 after every 10,000 steps. Gradient clipping by global norm was also utilized to enhance the stability of the training process if large gradients occurred during the training. A warm start approach for training using the physics-based losses was used to reduce training time and cost, where the network is trained on data for the first part of the training, then the physics loss was introduced to the loss function.

A.1.1 Domain Decomposition

The domain decomposition process is defined as breaking the overall solution domain into several subdomains and defining a model for each subdomain. As discussed by Jagtap et al. (2020) and Hu et al. (2022), breaking down the overall solution domain into subdomains can allow for better generalization capability for complex multiphysics and multiscale problems since predictions for each subdomain are performed on the subnetwork of that domain. This step is also crucial to enable the addition of interface boundary conditions to the loss function, which for the energy equation consists of continuity of flux and continuity of temperature.

Although there are several ways of implementing domain decomposition based on the problem being solved, in this work we focus on applying it to thermal surrogates only. This is so that we can apply the interface boundary conditions [Eqs. (19) and (20)] and better capture the temperature field across different bodies that are a part of the same simulation, separated by interfaces. Figure A1 shows an example of domain decomposition for thermal surrogates.

A.1.2 Fourier Feature Embeddings: Continued

Continuing from Section 3.3, a comparison between predictions with and without Fourier feature embeddings, for flow over a small cylinder with a vertical axis, is shown in Fig. A2. The solution that utilizes the embedding has much more clearly defined flow structures and capturing of the boundary of the object. The time taken per training iteration when using the embedding is 1.2× the time taken without the embedding.

A.2 Design Optimizer Setup

To optimize physical designs with trained models that predict flow thermal results, the particle swarm optimization (PSO) (Kennedy and Eberhart, 1995) algorithm is used. It is a zero-order

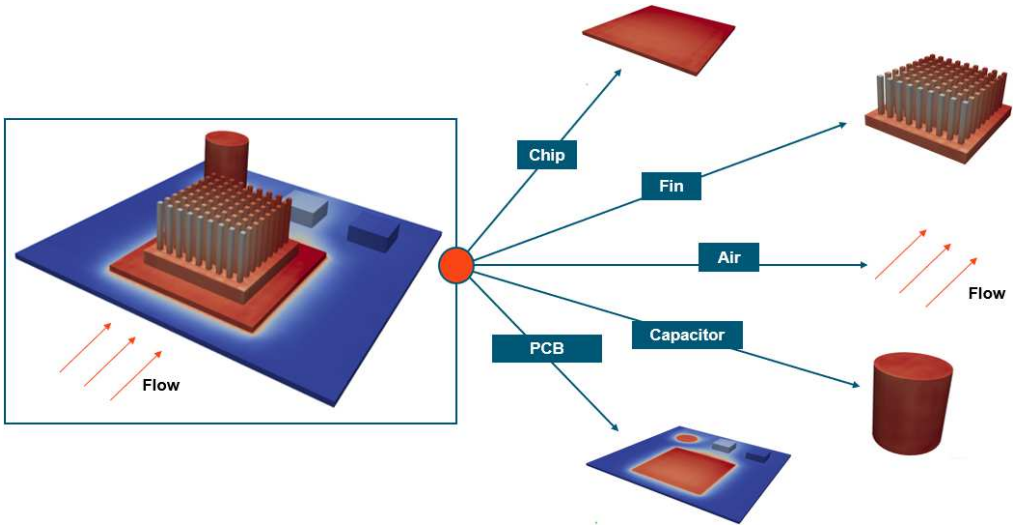


FIG. A1: Graphic showing the domain decomposition process for a printed circuit board thermal surrogate modeling problem. Every part of the solution domain depicted has a different neural network model predicting the solution field in it.

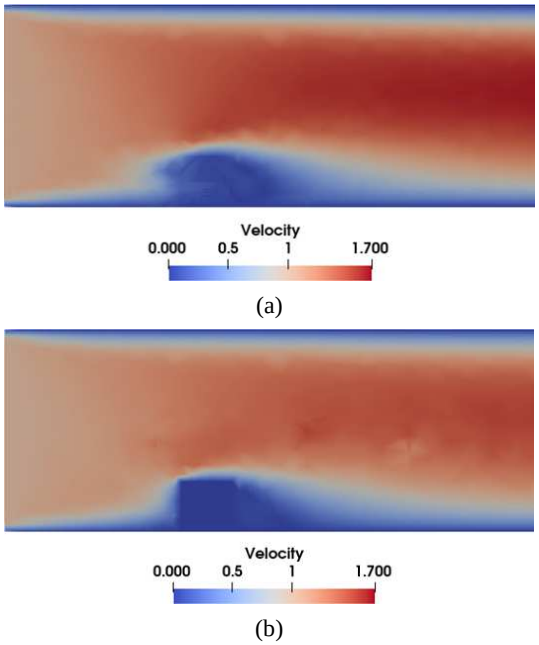


FIG. A2: 2D cut plane of the prediction of flow over a cylinder, with and without Fourier feature embedding: (a) no Fourier feature embedding, (b) with Fourier feature embedding

optimization method that uses an initial distribution of particles in the search space and, based on the "fitness" of each particle computes, new positions and velocities of particles in the search space. Eventually, the particles converge toward the optima.

The current categories of problems being solved in this paper (i.e., constrained flow-thermal design optimization problems) take the general form

$$\min_{\mathbf{u}} f(\mathbf{u}), \tag{A.1}$$

s.t.

$$g_i(\mathbf{u}) \leq X_i, \quad i = 1 \ldots N, \tag{A.2}$$

$$u_j^{\min} \leq u_j \leq u_j^{\max}, \quad j = 1 \ldots M, \tag{A.3}$$

where f represents an objective function, g_i represents the ith constraint, and X_i represents constraint values. $\mathbf{u}$ represents the input vector of design parameters (of length M), and each component of $\mathbf{u}$ has a $u_j^{\min}$ and $u_j^{\max}$ that they can take. The objective and constraint values would be a derivative of flow thermal variables such as pressure, temperature, or velocities, or design variables, such as geometry lengths, inflow rates, or source terms. The objective and constraints can be placed on bodies, individual surfaces, or the internal volumes of components that are a part of the simulation.

To solve this problem via the PSO algorithm, the constrained optimization problem is converted to an unconstrained problem via the penalty method to get the objective function

$$f(\mathbf{u}) + \sum_{i=1}^{N} \lambda_i \cdot \big(g_i(\mathbf{u}) - X_i\big)^2 \cdot 1\big(g_i(\mathbf{u}) > X_i\big) + \beta \sum_{i=1}^{N} 1\big(g_i(\mathbf{u}) > X_i\big). \tag{A.4}$$

The first term is the constrained objective function. The second term represents the degree of deviation of the constraint from the boundary if the constraint is violated. The third term adds cost based on the number of constraints violated, and λ_i and β are constants.

Once the objective function is set up, the ith particle positions ($\mathbf{u}$) and velocities are updated according to the equations

$$\mathbf{u}^i = \mathbf{u}^i + \mathbf{v}^i, \tag{A.5}$$

$$\mathbf{v}^i = w\mathbf{v}^i + c_1 r_1\big(\mathbf{u}^i_{\text{best}} - \mathbf{u}^i\big) + c_2 r_2\big(\mathbf{u}_{\text{best}} - \mathbf{u}^i\big), \tag{A.6}$$

where c_1, c_2, r_1, r_2, and w are constants.

Figure A3 depicts the design cycle using the PSO algorithm. In traditional design optimization, step 1 is done using CFD solvers, but this can get very expensive. NN–based surrogates are an ideal tool to replace CFD solvers for early-stage design cycles.

A.3 Design of Experiment (DoE) Space Sampling

In this work, the optimal sampling of the DoE space is done using the Morris–Mitchell (Morris and Mitchell, 1995) space-filling criterion. N sets of points are created, and each set has m randomly chosen points in the DoE space. More specifically, if $\mathcal{S}$ depicts the set of all sets of points, $s_i \in \mathcal{S}$ s.t.

$$s_i = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_j\}, \quad i = 1 \ldots N, \quad j = 1 \ldots m, \tag{A.7}$$

where $\mathbf{x}_j$ represents the coordinates of point j in the DoE space.

For each s_i, the Morris–Mitchell criterion score is computed:

$$\Phi_i = \left(\sum_j d_j^{-q} \right)^{1/q}, \tag{A.8}$$

where d_j represents the pairwise distance between every point in the set s_i, and q is a fixed parameter controlling the spacing between points. The set with the minimum Φ_i is considered the most space-filling sampling of the DoE space. Optimally space-filled sampling strategies are crucial to the cost-effective creation of accurate surrogate models. Design space sampling in computer experiments, and the resulting effects on the accuracy of the generated surrogate models, is a well-studied topic (Afzal et al., 2017; Lin et al., 2001). Moreover, the effects of design space sampling for physics-informed deep learning have also been studied by Das and Tesfamariam (2022) for a variety of PDEs. In the context of PINNs, the effects of sampling the physical space for collocation points have also been extensively studied (Wu et al., 2023).

A.4 Extended: Surrogate Modeling and Design Optimization of a Heat Sink via PINNs

A.4.1 DoE Tables

Tables A1 and A2 show the parameters used to train and query the model. The data for the training points was generated by running Altair AcuSolve®. Each solution took roughly 15 minutes on a four-core machine.

A.4.2 Cost Benefit Analysis of Surrogate Model Compared to CFD-Based Optimization

A quantitative comparison of PINN surrogate versus CFD-based optimization is shown. For the PINNs model, training was performed on a single Titan V GPU card.

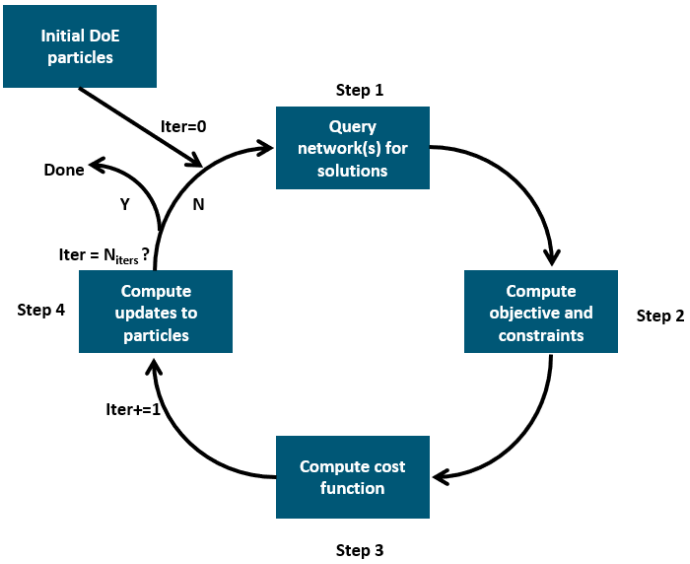


FIG. A3: A design iteration using a design optimization algorithm (like PSO)

TABLE A1: DoE table to train the surrogate model

No.	Inflow velocity (m/s)	Fin height (mm)	Power generated (W)
1	5	19	45
2	3	15	30
3	3	15	60
4	3	23	30
5	3	23	60
6	7	15	30
7	7	15	60
8	7	23	30
9	7	23	60
10	5.03	15.3	59
11	3.01	18.4	45.55
12	6.94	19.4	36.05
13	6.53	22.6	51

TABLE A2: Test points to query the model

No.	Inflow velocity (m/s)	Fin height (mm)	Power generated (W)
Test point 1	6	20	47.5
Test point 2	4	17	40
Test point 3	6.5	22	55
Test point 4	3.5	19	50

Table A3 shows the cost-benefit analysis of the hybrid PINNs-CFD surrogate model versus CFD for the heat sink design optimization problem shown in Section 4.3. The comparison visualized in Fig. A4 shows the total time taken to optimize the design using the PSO method and compares the time taken versus the number of iterations. The "model training time" encapsulates the time taken to create the data and train the model so that it is a fair comparison.

TABLE A3: Comparing design iteration times using CFD versus ANN surrogate for the heatsink problem

Solve Type	Model training time	Time for a design iteration	Time for 10 design iterations
CFD (4 cores)	—	160 min	1600 min
PINN (1 GPU + 1 core)	286 mins	55 seconds	300 mins

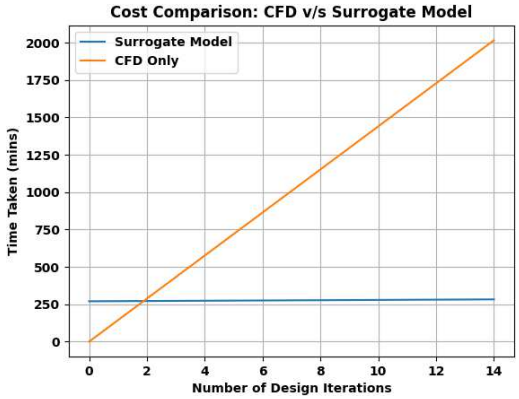


FIG. A4: Time comparison of doing PSO-based design iterations using the surrogate versus CFD. Once the surrogate returns a truncated DoE space, the designer can perform high-fidelity CFD to fine tune the design.